

AutoCrew: Automatic Generation of Agentic AI Crews for Financial Modeling Tasks

Sarim Shah, Moiz ul Haq, Muhammad Rouhan

Department of Computer Science, FAST–National University of Computer and Emerging Sciences, Karachi, Pakistan

Supervisor: Dr. Shakil Ahmed

{k224299, k224587, k224577}@nu.edu.pk

Repository: <https://github.com/ssarimm/AutoCrew-Meta-Agent-Research>

Abstract—The emergence of Large Language Models (LLMs) has enabled a new class of multi-agent AI frameworks capable of automating complex multi-stage workflows in the financial domain. Despite demonstrated effectiveness, existing systems such as CrewAI, AutoGen, and MetaGPT require developers to manually define every agent, task, tool assignment, and dependency in source code for each new use case—an approach that is time-consuming, rigid, technically demanding, and inaccessible to non-technical domain experts. This paper presents AutoCrew, a framework that automatically generates and orchestrates AI agent crews from simple natural language task descriptions, directly addressing the "Orchestration Bottleneck" identified as an open research problem by Okpala et al. [1]. A central Meta Agent interprets the user's objective through a five-step pipeline—task decomposition, agent selection, tool assignment, instruction writing, and JSON generation—producing a fully executable crew configuration in approximately 6 seconds on Groq (Llama 3.3 70B Versatile). AutoCrew was evaluated against a manually coded baseline (Path A) across three publicly available financial datasets—UCI Credit Card Approval, Kaggle Fraud Detection 2023, and Give Me Some Credit—using a controlled dual-path architecture that preserves identical execution infrastructure across both paths, ensuring that performance differences are attributable solely to the crew generation method. Results demonstrate that AutoCrew achieves accuracy within 0.21% of the manual baseline across all three datasets and receives identical MRM APPROVED compliance verdicts in every experiment, while reducing one-time developer setup time from approximately 300 minutes to under 6 seconds—a reduction exceeding 99.9%. On the Give Me Some Credit dataset, AutoCrew's total wall-clock execution time was 42% faster than the manual baseline, demonstrating that auto-generated crews can also outperform hand-crafted configurations on execution efficiency. Dataset-aware preprocessing injection automatically handles the distinct data characteristics of each supported financial dataset, including missing value imputation, class imbalance correction, and column header conventions. This work contributes a practical, reproducible, and open-source step toward democratizing advanced financial AI, enabling non-technical domain experts—financial analysts, risk managers, and compliance officers—to command sophisticated multi-agent pipelines through natural language alone.

Index Terms—Agentic AI, Multi-Agent Systems, Financial Modeling, Large Language Models, Workflow Automation, Model Risk Management, CrewAI, Natural Language Processing, Automated Pipeline Generation, Orchestration Bottleneck.

I. INTRODUCTION

Artificial Intelligence has become an indispensable component of modern financial services, underpinning applications ranging from credit risk assessment, fraud detection, and regulatory compliance to portfolio optimization, stress testing, and automated documentation. The scale and complexity of these applications have outpaced what traditional monolithic machine learning models can efficiently address. Single-model pipelines operate as black-box units, offer limited transparency, cannot natively coordinate across multiple analytical stages, and must be retrained from scratch when task requirements evolve. Financial institutions increasingly require end-to-end analytical pipelines that integrate data ingestion, exploratory analysis, feature engineering, model training, performance validation, stress testing, and regulatory compliance reporting in a single coherent workflow.

The introduction of Large Language Models has given rise to a powerful new paradigm: multi-agent systems (MAS) in which a team of specialized AI agents, each assigned a distinct role and toolset, collaborates to accomplish complex tasks through sequential or hierarchical execution [1][2]. Frameworks such as CrewAI [3], AutoGen [17], and MetaGPT [10] have demonstrated the practical power of role-based agent decomposition, where distinct agents handle data analysis, feature engineering, model training, compliance checking, and documentation—mirroring how real-world financial organizations operate.

This collaborative structure provides improved transparency, modularity, and auditability over single-model pipelines, making it particularly well-suited to financial services environments that must satisfy Model Risk Management (MRM) requirements.

The base paper by Okpala, Golgoon, and Kannan [1] applied this paradigm directly to financial services, constructing manually coded CrewAI crews for credit scoring, fraud detection, and portfolio credit risk modeling. Their results established that multi-agent pipelines outperform single-model baselines in transparency and workflow modularity, and they validated that agentic AI is a viable and practical approach for the financial domain. However, the paper explicitly identified automated crew generation as an open research problem: every time a new financial task arises—a different dataset, a new classification objective, or a new compliance requirement—a skilled developer must rewrite significant portions of the agentic codebase. This is not scalable.

This fundamental limitation, which we term the *Orchestration Bottleneck*, manifests in three concrete ways. First, agent definitions—including role, goal, backstory, and tool assignment—must be manually coded for every new task. Second, task instructions must be hand-written with sufficient detail for the LLM to follow correctly, a process requiring both financial domain knowledge and CrewAI framework expertise. Third, the ordering and dependency structure of tasks must be explicitly designed by a developer who understands both the financial domain and the CrewAI execution model. The combination of these requirements creates a barrier that limits multi-agent financial AI to organizations with dedicated ML engineering teams, entirely excluding non-technical domain experts such as financial analysts, risk managers, and compliance officers.

This paper presents AutoCrew, a framework that eliminates the *Orchestration Bottleneck* by automatically generating and executing AI agent crews from natural language task descriptions. The user simply describes the financial modeling objective in a single sentence—for example, "Build a credit risk model using this dataset"—and AutoCrew's Meta Agent produces a complete, executable crew configuration in approximately 6 seconds. A Crew Factory then instantiates the crew at runtime with no code changes required. The system was evaluated across three financial datasets against a manually coded baseline using a controlled dual-path architecture, confirming that auto-generated crews achieve comparable performance and identical MRM compliance outcomes while eliminating nearly all developer setup overhead.

The main contributions of this paper are:

- A five-step Meta Agent pipeline that converts a natural language task description into a complete, executable JSON crew configuration (`crew_config.json`) in approximately 6 seconds on Groq (Llama 3.3 70B Versatile), with deterministic rule-based fallbacks for all five steps.
- A Crew Factory that dynamically instantiates live CrewAI Agent, Task, and Crew objects from the generated JSON at runtime, requiring zero code changes per new task and achieving complete decoupling of crew design from crew execution.
- A controlled Tool Registry that restricts agents to three verified tools (`code_execution`, `eda_tool`, `cag_tool`), preventing hallucination of non-existent libraries at runtime and ensuring every generated crew executes without runtime errors.
- A dual-path evaluation architecture (Path A: Manual Baseline from Okpala et al. [1]; Path B: AutoCrew) enabling controlled, fair side-by-side comparison on three financial datasets using identical execution infrastructure, ensuring performance differences are attributable solely to the crew generation method.
- Dataset-aware preprocessing injection that automatically handles the distinct data characteristics of each supported financial dataset—missing value imputation, class imbalance correction, and column header conventions—eliminating an entire class of preprocessing bugs that previously required manual intervention per dataset.
- An integrated MRM Crew providing automated Model Risk Management compliance verdicts, which receives APPROVED status on all three evaluated datasets for both paths.
- Quantitative demonstration of a $23\times$ – $50\times$ reduction in time-to-first-result across all three datasets when total developer effort is included, with AutoCrew completing setup in under 6 seconds versus approximately 300 minutes for the manual baseline.

The remainder of this paper is organized as follows. Section II reviews related work on multi-agent systems, agentic AI in financial services, and LLM-based workflow automation. Section III describes the system architecture. Section IV presents the Meta Agent pipeline in detail. Section V describes the Tool Registry and sandboxed execution. Section VI covers the experimental setup. Section VII presents and discusses results across all three datasets. Section VIII compares the architectures. Section IX discusses limitations. Section X outlines future work, and Section XI concludes.

II. RELATED WORK

A. Multi-Agent Systems with LLMs

LLM-based multi-agent systems have attracted significant research attention in recent years, driven by the observation that complex tasks can be more effectively addressed by a team of specialized agents than by a single monolithic model. Tran et al. [2] provide a comprehensive survey of collaboration mechanisms in MAS, establishing taxonomies for agent communication, memory management, and task decomposition strategies. Their work identifies three primary collaboration patterns—centralized orchestration, decentralized peer-to-peer, and hierarchical delegation—all of which inform AutoCrew’s architectural decisions. In centralized orchestration, a manager agent delegates subtasks to worker agents; in decentralized peer-to-peer, agents communicate directly without a central coordinator; in hierarchical delegation, teams of agents are organized in tree structures with recursive task decomposition.

AutoGen [17] introduces conversational multi-agent pairs that iteratively refine solutions through natural-language dialogue, enabling agents to critique and improve each other’s outputs without explicit task decomposition. MetaGPT [10] encodes software engineering role conventions into agents and enforces structured communication through document artifacts, producing deliverables that span design documents, code, and test cases. Crawford et al. [11] demonstrate how multi-agent collaboration can be deployed at production scale for enterprise task automation, covering diverse domains from customer service to data analysis. Duan and Wang [16] further explore hybrid LangGraph and CrewAI orchestration, demonstrating how combining graph-based control flow with role-based execution can reduce development overhead in multi-stage financial pipelines.

CrewAI [3], which serves as AutoCrew’s execution backbone, provides production-grade role-based sequential and hierarchical agent execution with native tool integration. Its ability to specify agent roles, goals, backstories, and tool assignments in a declarative manner, and to chain tasks through context passing, makes it well-suited as an execution target for a generative configuration pipeline. Importantly, while all of these frameworks are powerful orchestration engines, none provides a mechanism for automatically generating the crew structure itself from a high-level user objective. The developer is always required to pre-define the entire crew configuration in code—the fundamental gap that AutoCrew addresses.

B. Agentic AI in Financial Services

The application of agentic AI to financial services has expanded rapidly following demonstrations that LLM-powered agents can perform domain-specific reasoning, execute computational tools, and generate structured documentation. The foundational work by Okpala, Golgoon, and Kannan [1] is the direct precursor to AutoCrew. Their paper applies manually constructed CrewAI crews to three financial tasks—credit card fraud detection, credit card approval prediction, and portfolio credit risk modeling—demonstrating that multi-agent architectures offer superior transparency, workflow structure, and auditability compared to single-model baselines. The Modeling Crew in their system consists of a Manager, Data Analyst, ML Engineer, and Risk Analyst, while the MRM Crew consists of a Manager, Compliance Officer, Model Validator, and Chief Risk Officer. Their results validated that agentic AI is a viable and practical approach for financial services. Critically, the paper explicitly identified automated crew generation as future work, which AutoCrew directly addresses.

Beyond the base paper, several other systems have applied agentic AI to financial domains. Li et al. [12] propose QuantAgents, a multi-agent system for quantitative financial simulation where specialized agents analyze news, assess technical indicators, and manage portfolio risk in coordinated trading simulations. Yu et al. [13] present FinCon, an LLM-based multi-agent system that employs conceptual verbal reinforcement—agents critiquing and reinforcing each other’s financial decisions—to improve decision-making quality across tasks including portfolio management, algorithmic trading, and financial question answering. Yuan et al. [15] demonstrate Alpha-GPT 2.0, which introduces Human-in-the-Loop (HITL) patterns for quantitative investment AI, emphasizing that human oversight at key decision points improves both safety and performance in financial pipelines. This HITL pattern is directly incorporated into AutoCrew’s architecture between the Modeling Crew and MRM Crew.

Despite the breadth of these contributions, all existing financial AI agent systems share a common limitation: they require hand-coded agent definitions for each new task or use case. The analyst or developer must understand both the financial domain and the agentic framework’s API to deploy a new workflow. AutoCrew is the first system to fully automate this process, allowing a non-technical financial professional to initiate a complete multi-agent financial AI pipeline from a single natural language description.

C. Workflow Automation and LLM Planning

Research in LLM-based planning demonstrates that language models can decompose complex goals into subtask sequences, though doing so reliably and with structured outputs remains an active challenge. Ferrag et al. [4] survey the progression from LLM reasoning to autonomous agentic behavior, noting that task planning, dynamic tool selection, and reliable tool use remain central open challenges for production deployment. Their survey identifies the progression from chain-of-thought prompting through ReAct-style tool use to fully autonomous multi-step agent systems, with each level requiring increasingly robust planning and error recovery mechanisms. Nisa et al. [5] review the broader agentic AI landscape, identifying orchestration flexibility—the ability to dynamically adapt agent structure to task requirements—as a key prerequisite for real-world deployment.

Hua et al. [14] address safety and trustworthiness in LLM-based agentic systems in TrustAgent, highlighting hallucination of non-existent tools and APIs as a significant risk that must be explicitly mitigated in production systems. This observation directly motivates AutoCrew's Tool Registry design: by restricting the LLM to a typed registry of verified tools and validating all tool name references at instantiation time, AutoCrew prevents this class of failures before any agent executes. LangChain [6] provides a widely-used framework for building LLM-powered applications through composable chains and provides an alternative execution model to CrewAI; AutoCrew's design is independent of the underlying execution framework and could be extended to generate LangChain configurations in future work.

AutoCrew's combination of LLM-driven generation with deterministic rule-based fallbacks distinguishes it from purely LLM-dependent automation approaches. Every one of the five pipeline steps has a fallback that activates on API timeout, rate limit, or malformed JSON response, ensuring the system produces a valid executable configuration even when the LLM is unavailable. This hybrid design ensures reliability in production financial environments where API availability cannot be guaranteed.

D. Positioning Summary

Table I summarizes how AutoCrew positions relative to key related systems across seven dimensions: automated crew generation, financial domain focus, MRM compliance integration, controlled tool execution, no-code task adaptation, and open-source availability.

TABLE I. Positioning of AutoCrew Relative to Key Related Systems

System	Auto Generation	Financial Focus	MRM Integration	Tool Control	No-Code Adaptation	Open-Source
CrewAI (base) [3]	No	No	No	Manual	No	Yes
Okpala et al. [1]	No	Yes	Yes	Manual	No	No
AutoGen [17]	No	No	No	Partial	No	Yes
MetaGPT [10]	Partial	No	No	No	No	Yes
LangChain [6]	No	Partial	No	Partial	No	Yes
QuantAgents [12]	No	Yes	No	Manual	No	No
FinCon [13]	No	Yes	No	Manual	No	No
AutoCrew (This Work)	Yes	Yes	Yes	Registry	Yes	Yes

III. REQUIREMENTS

A. Functional Requirements

The functional requirements for AutoCrew were derived from the core problem statement and the limitations identified in Okpala et al. [1]. They specify what the system must do to demonstrate that automated crew generation is both feasible and practical for financial AI workflows.

- The system shall accept a natural language financial task description, a dataset file path, and an LLM provider selection as inputs via CLI.
- The system shall support three execution modes: manual (Path A only), auto (Path B only), and compare (both paths with automated comparison report).
- The Meta Agent shall decompose the task, select agent roles, assign tools, write task instructions, and produce a valid `crew_config.json` file in under 10 seconds under normal Groq API conditions.
- All five pipeline steps shall include rule-based fallbacks that activate automatically on LLM failure, malformed JSON, or timeout, ensuring the system produces a valid configuration under all conditions.
- The Crew Factory shall instantiate live CrewAI Agent, Task, and Crew objects from the generated JSON without any code changes per task.
- The system shall execute a Modeling Crew followed by an MRM Crew, with a Human-in-the-Loop gate between them allowing optional human review before the compliance audit proceeds.
- The system shall automatically detect the dataset by filename substring and inject the correct preprocessing logic for each supported financial dataset.
- The system shall extract accuracy, F1, precision, and recall from raw agent text output and save results as JSON for reproducibility.
- The system shall generate comparison charts and a multi-page PDF report when run in compare mode.

B. Non-Functional Requirements

Non-functional requirements specify the quality attributes the system must satisfy to be considered production-ready and suitable for financial compliance environments.

- Under normal Groq API conditions, the Meta Agent pipeline shall complete crew configuration in under 10 seconds.
- Code execution shall be sandboxed via subprocess isolation with a 120-second timeout to prevent runaway scripts and protect the host environment.
- The Tool Registry shall be extensible to new verified tools without modifying any existing agent logic.
- All generated crew configurations shall be version-stamped with a timestamp and saved to `configs/generated/` for audit traceability and reproducibility.
- The system shall handle Groq API rate limits gracefully using the configurable `AUTOCREW_PHASE_WAIT` environment variable, with a default wait between pipeline phases.
- The system shall support a Mock LLM mode for complete end-to-end testing without any API key dependency.
- The system shall produce deterministic outputs under identical inputs in Mock LLM mode, enabling reproducible research comparisons.

C. Technology Stack

Table II summarizes the technology stack used in AutoCrew, including the rationale for each component selection.

TABLE II. AutoCrew Technology Stack

Component	Technology / Library	Version / Notes	Rationale
Agentic Framework	CrewAI	Latest (Apr. 2026)	Production-grade role-based sequential and hierarchical agent execution
Primary LLM	Groq — Llama 3.3 70B Versatile	Free tier, 12K TPM	High throughput, low latency, cost-free inference for evaluation
Alternative LLMs	Google Gemini 2.0 Flash / Ollama / Mock LLM	Provider-selectable	Flexibility, offline testing, and provider-level performance comparison
Language	Python 3.10+	3.10+	Full ecosystem compatibility with all ML and CrewAI libraries
ML / Data Libraries	scikit-learn, pandas, numpy	Latest stable	Standard financial data science stack; Random Forest, metrics, preprocessing
Configuration Format	JSON	Native	Human-readable, version-controlled crew configuration persistence
PDF Report Generation	ReportLab	Latest	Automated multi-page comparison report with embedded charts
Charting	matplotlib	Latest	Five evaluation figures per run; saved to figures/ directory
Version Control	Git / GitHub	Public repo	Full reproducibility; open-source at github.com/ssarimm/AutoCrew-Meta-Agent-Research

IV. SYSTEM ARCHITECTURE

AutoCrew is structured around a dual-path evaluation architecture that allows a directly controlled comparison between the manually coded baseline from Okpala et al. [1] and the automated system. Both paths execute on the identical CrewAI engine, use the same tool implementations, traverse the same Human-in-the-Loop gate, and receive dataset preprocessing from the same injection mechanism, ensuring that any observable performance difference is attributable solely to the crew generation method, not differences in execution infrastructure. The system architecture is organized into four horizontal layers, each with a distinct responsibility.

A. User Interface Layer

The entry point (`src/main.py`) accepts a natural language task description, the dataset path, an LLM provider selection, and an execution mode via CLI. Three execution modes are supported. In manual mode, the system invokes the hardcoded Path A crew and runs the Modeling and MRM Crews without any Meta Agent involvement. In auto mode, the Meta Agent pipeline generates the `crew_config.json`, the Crew Factory instantiates the crew, and execution proceeds. In compare mode, both paths execute sequentially on the same dataset, timing is recorded for each phase, and the evaluation infrastructure generates a side-by-side comparison report including five matplotlib charts and a multi-page PDF summary. Four LLM providers are supported: Groq (Llama 3.3 70B Versatile, the recommended default), Google Gemini 2.0 Flash, Ollama (local inference), and a Mock LLM for complete end-to-end testing without any API key dependency.

B. Meta-Agent Layer

The Meta Agent (`src/meta_agent/`) is the core innovation of AutoCrew. It processes the user's natural language input through a five-step sequential pipeline, each step issuing one structured LLM call with JSON-constrained output conditioned on all prior step outputs. The pipeline produces a structured `crew_config.json` file in approximately 6 seconds on Groq. A deterministic rule-based fallback is integrated into every pipeline step, activating automatically when the LLM call fails due to API rate limits, network timeouts, malformed JSON responses, or other errors. This hybrid design ensures the pipeline remains fully operational even under adverse API conditions. Generated configurations are saved to `configs/generated/` with a timestamp, providing versioned crew history and full audit traceability for compliance purposes.

C. Orchestration Layer: Crew Factory and Tool Registry

The Crew Factory (`src/crew_factory/crew_builder.py`) acts as the critical bridge between the declarative JSON configuration produced by the Meta Agent and the imperative CrewAI Python API. By completely decoupling crew design from crew execution, the factory ensures that any valid `crew_config.json`—whether generated autonomously by the Meta Agent, manually authored by a developer, or produced by future generative models—can be instantiated and executed without modifying the underlying source code. The factory implements the following sequence: configuration loading and structural validation; tool name resolution through the Tool Registry; dynamic CrewAI Agent object construction; Task object instantiation with dependency mapping via `context_from_indices`; and final Crew assembly with the specified process logic.

The Tool Registry (`src/tools/tool_registry.py`) is a strictly typed name-to-instance mapping that serves as the sole source of truth for available tools. The LLM receives only the names and descriptions of registered tools during the tool assignment step, and the Crew Factory resolves all tool name strings to verified Python instances at instantiation time. Any tool name not found in the registry raises a registry error before any agent executes, preventing the class of silent runtime failures caused by LLM hallucination of non-existent libraries identified as a key risk by Hua et al. [14].

D. Execution and Evaluation Layer

The Modeling Crew and MRM Crew execute sequentially, separated by a Human-in-the-Loop (HITL) gate that pauses execution between the two phases. The gate supports two configuration modes: auto-approve (bypassed for fully automated pipelines) and require-confirmation (pauses execution and requests explicit human input before the MRM Crew begins). This pattern mirrors the HITL workflow described by Yuan et al. [15] for quantitative investment AI, adapting it to the compliance audit context. The evaluation infrastructure (`src/evaluation/`) implements four modules: `MetricsCollector` for extracting accuracy, F1, precision, and recall from raw agent output text using regex patterns; `comparison.py` for orchestrating both-path sequential execution and recording timing; `generate_charts.py` for producing five matplotlib figures; and `report_generator.py` for assembling a multi-page PDF comparison report using ReportLab.

E. Dual-Path Architecture Comparison

Table III presents a detailed comparison of Path A and Path B across all major architectural dimensions, highlighting the practical differences in how the two approaches construct and execute financial AI crews.

TABLE III. Detailed Dual-Path Architecture Comparison

Dimension	Path A: Manual Baseline (Okpala et al.)	Path B: AutoCrew (This Work)
Agent Definition	Hardcoded in Python source files	Generated by Meta Agent LLM from natural language
Task Instructions	Hand-written in Python source files	Auto-generated from user's one-sentence task description
Tool Assignment	Manually specified in code per agent	LLM selects from controlled Tool Registry; validated at runtime
New Task Adaptation	Rewrite Python class files from scratch	Change one sentence; Meta Agent regenerates full configuration
Configuration Saved	No persistent record; lives only in source code	Yes — <code>crew_config.json</code> (versioned, timestamped, auditable)
Crew Size	3 Modeling + 3 MRM = 6 agents (fixed)	Dynamic — LLM decides (typically 4 Modeling + 3 MRM)

		= 7 agents)
Dataset Preprocessing	Manually coded per dataset	Auto-injected by Instruction Writer (Step 4 of pipeline)
Execution Engine	CrewAI (identical)	CrewAI (identical)
HITL Gate	Present (identical)	Present (identical)
MRM Crew	Manually coded	Auto-generated
One-Time Setup Cost	~300 min skilled developer effort	~6 seconds automated Meta Agent configuration
MRM Verdict (all datasets)	APPROVED (all 3)	APPROVED (all 3)
Max Accuracy Gap	Baseline	$\leq 0.21\%$ below baseline

V. META-AGENT PIPELINE DESIGN

The Meta Agent is the core contribution of AutoCrew. It operates as a five-step sequential pipeline, each step making an independent structured LLM call that produces JSON output conditioned on all prior step outputs. The pipeline transforms a single natural language sentence into a complete, executable multi-agent crew configuration in approximately 6 seconds on Groq (Llama 3.3 70B Versatile). Prompt engineering was critical to achieving consistent JSON output across different LLM providers: each prompt explicitly defines the output format via concrete JSON examples and mandates the exclusion of conversational filler, separating generation logic from task-specific instructions and allowing granular control over agent personas and tool assignments without compromising framework stability. Table IV summarizes all five steps.

TABLE IV. Meta Agent Five-Step Pipeline (~6 s total on Groq Llama 3.3 70B Versatile)

Step	Module	Input	Output	Avg. Time	Fallback Strategy
1	task_decomposer.py	NL task description + dataset path	Ordered subtask list with dependency graph	~1.0 s	Keyword-based template selection (3+2 fixed structure)
2	agent_selector.py	Subtask list	Agent role, goal, and backstory per subtask	~1.0 s	Direct lookup in agent_templates.json registry
3	tool_assigner.py	Agent list + tool catalog names	Tool name list per agent from registry	~1.0 s	Role-based default assignments (hardcoded mappings)
4	instruction_writer.py	Full context + dataset path	Detailed dataset-aware task descriptions + expected outputs	~2.2 s	Template + dataset-specific preprocessing hints injected
5	json_generator.py	All prior step outputs	Final crew_config.json with metadata, agents, tasks, dependencies	~0.5 s	Direct schema assembly from prior step outputs

A. Step 1 — Task Decomposition

The Task Decomposer (src/meta_agent/task_decomposer.py) receives the raw natural language task description and dataset path and issues a single structured LLM call that produces an ordered list of subtasks with explicit dependencies. The prompt enforces structural constraints that reflect financial modeling best practices: at most four Modeling Crew subtasks and three MRM Crew subtasks, a mandatory model training step, a mandatory stress testing step, and a final verdict step. These constraints prevent the LLM from generating over-specified or under-specified crew structures that would fail during CrewAI execution. The module also enforces that each subtask specifies which prior subtask outputs it depends on, enabling the Crew Factory to construct the correct context_from_indices dependency graph at instantiation time.

The deterministic fallback for this step returns a hardcoded but complete structure: three Modeling subtasks (data loading and EDA, model training and evaluation, model card documentation) and two MRM subtasks (compliance and stress testing, final

verdict), with the user's task description embedded in the training subtask's description field. This structure is derived from the Okpala et al. [1] baseline and represents a known-good crew design for general financial classification tasks. In all experiments reported in this paper, the LLM successfully generated custom subtask decompositions without triggering the fallback.

B. Step 2 — Agent Selection and Persona Assignment

Given the subtask list produced by Step 1, the Agent Selector (`src/meta_agent/agent_selector.py`) assigns each subtask an agent role with a specific persona consisting of a role title, goal statement, and backstory narrative. Roles are drawn from a role template registry (`registries/agent_templates.json`) that encodes a financial AI agent persona ontology, including Data Analyst, Data Scientist, Senior Data Scientist, and ML Engineer for the Modeling Crew, and Compliance Officer, Stress Testing Engineer, and Chief Risk Officer for the MRM Crew. The LLM generates agent backstories by combining role conventions from the registry with task-specific context, conditioning the agent's reasoning behavior within CrewAI. The rule-based fallback assigns predefined backstories directly from the registry when the LLM call fails, ensuring the crew always has well-formed agent personas.

The backstory is particularly important in the CrewAI execution model because it conditions the LLM's persona for that agent's reasoning steps. A Compliance Officer agent with a backstory describing regulatory expertise in credit risk will produce more contextually appropriate MRM assessments than one with a generic backstory. The Agent Selector's LLM-generated backstories capture task-specific nuances—for example, noting that the agent is reviewing a fraud detection model trained on highly imbalanced data—that generic templates would miss.

C. Step 3 — Tool Assignment and Validation

The Tool Assigner (`src/meta_agent/tool_assigner.py`) maps each agent to tools from the controlled Tool Registry. The LLM receives only the names and descriptions of registered tools (served from `registries/tool_catalog.json`), preventing it from referencing external libraries or hallucinated tool names. The expected assignment pattern for financial classification tasks is: `code_execution` to the Data Analyst and ML Engineer agents for data loading, model training, and evaluation; `eda_tool` to the Data Scientist agent for exploratory analysis; and `cag_tool` to the Compliance Officer agent for MRM assessment. The rule-based fallback implements this pattern directly as a role-to-tool mapping dictionary, ensuring correct assignments even when the LLM call fails.

At execution time, the Crew Factory resolves all tool name strings to verified Python instances via the registry using a name-to-instance lookup. Any tool name not found in the registry raises a `RegistryError` before any agent executes—a "fail fast" design that prevents hard-to-debug runtime failures deep in agent execution chains. This approach directly addresses the tool hallucination risk identified by Hua et al. [14] as a primary safety concern in autonomous agent systems.

D. Step 4 — Dataset-Aware Instruction Writing

The Instruction Writer (`src/meta_agent/instruction_writer.py`) is the most computationally intensive pipeline step (~2.2 s on Groq) because it must synthesize the full prior context—subtask list, agent assignments, tool assignments, and dataset metadata—into detailed natural language task descriptions and expected output specifications for each agent. The instructions specify exactly what Python code each agent should write, what metrics to report, and what the expected output format should be. Instruction quality is critical because it directly determines whether the LLM-powered agents produce correct, executable code and properly structured outputs.

The module is dataset-aware across two layers. At the LLM call layer, the system prompt instructs the language model to apply correct dataset-specific preprocessing based on the dataset filename. At the deterministic fallback layer, the module maintains a `dataset_configs` dictionary keyed by filename substring patterns that directly injects hardcoded preprocessing code when the LLM call fails. Table V details the preprocessing injection logic for each supported dataset.

TABLE V. Dataset-Aware Preprocessing Injection by the Instruction Writer

Dataset	Detection Pattern	Injected Preprocessing	Rationale
UCI Credit Card Approval	"credit_card_approval" in path	header=None on CSV load; df.replace('?', np.nan); df.dropna(); pos_label='+' hint for metrics	No column headers in raw file; '?' used as missing value sentinel; string target labels incompatible with default sklearn metrics
Kaggle Fraud Detection 2023	"creditcard_2023" in path	Drop 'id' column before training; RandomForestClassifier(class_weight='balanced'); average='weighted' for all metrics	PCA-anonymous id column must be excluded; extreme 99.62/0.38 imbalance causes all-majority prediction without reweighting
Give Me Some Credit	"cs-training" in path	fillna(median) for MonthlyIncome and NumberOfDependents; target = SeriousDlqin2yrs; class_weight='balanced'; average='weighted'	Naive dropna() discards 20% of 150K rows; median imputation preserves data; 6.4% minority distress class requires reweighting

This automatic detection and injection eliminates an entire class of preprocessing bugs that previously required manual intervention per dataset. For the UCI dataset, the '?' placeholder handling is particularly subtle—naively loading the CSV without replacing '?' with NaN and calling dropna() produces a model trained on string-valued columns, causing cascading failures throughout the pipeline. For the Give Me Some Credit dataset, the fillna(median) injection is critical for data retention: a naive dropna() call discards approximately 30,000 rows (20% of the dataset), significantly reducing model quality. For the Fraud Detection dataset, the class_weight='balanced' injection is essential for preventing the classifier from achieving spurious 99.62% accuracy by predicting the majority class for all samples.

E. Step 5 — JSON Generation and Dynamic Crew Instantiation

The JSON Generator (src/meta_agent/json_generator.py) assembles all prior step outputs into a structured crew_config.json file. The configuration schema includes: a metadata block (generation timestamp, LLM provider, model name, task description, dataset path); a modeling_crew block with an agents array and tasks array; and an mrm_crew block with the same structure. Each task entry includes a task_id, description, expected_output, agent_id reference, and a context_from_indices array specifying which prior task outputs this task should receive as context. The Crew Factory reads this file and programmatically constructs live CrewAI Agent, Task, and Crew objects, using context_from_indices to build the dependency graph.

This design achieves complete decoupling of crew design from crew execution. The crew_config.json is a human-readable, version-controlled artifact that can be inspected, modified, stored, and reloaded independently of any code changes. Future extensions could include a crew configuration marketplace where pre-validated configurations for common financial tasks are shared across organizations, or a browser-based interface where non-technical users edit configurations visually before execution.

VI. TOOL REGISTRY AND SANDBOXED EXECUTION

A common and severe failure mode in autonomous agent systems is the invocation of non-existent tools, APIs, or Python libraries—a form of hallucination that causes silent or difficult-to-debug runtime errors deep in agent execution chains. Hua et al. [14] identify this as one of the primary trustworthiness challenges in LLM-based agentic systems and argue that explicit tool scope restriction is a necessary safeguard for production deployments. AutoCrew addresses this through a strictly typed Tool Registry: a name-to-instance mapping that is the sole source of truth for available tools in the system. Table VI summarizes all registered tools, their functions, and their safety mechanisms.

TABLE VI. AutoCrew Tool Registry — Registered Tools and Safety Features

Tool Name	Module	Primary Function	Assigned To	Key Safety Feature
code_execution	code_execution.py	Executes agent-generated Python via sandboxed subprocess (120 s timeout); captures stdout and stderr	Data Analyst, ML Engineer, Stress Testing Engineer	Pre-flight syntax check + pos_label hinting; finally block deletes temp_script.py
eda_tool	eda_tool.py	Dataset-aware automated EDA: shape, dtypes, missing values, class balance, top correlations; returns structured text summary	Data Scientist	Dataset-aware: header=None for UCI CSV; '?' placeholder detection
cag_tool	cag_tool.py	Evaluates model documentation text against 7 MRM criteria; returns APPROVED / CONDITIONAL APPROVAL / REJECTED verdict with checklist	Compliance Officer	Keyword-based audit with percentage compliance score

A. Code Execution Tool

The `code_execution` tool (`src/tools/code_execution.py`) is the primary computational workhorse of the AutoCrew system. It facilitates the execution of agent-generated Python scripts via subprocess isolation, writing the script to a temporary file, executing it with a configurable timeout (defaulting to 120 seconds), and capturing both stdout and stderr. The tool implements three key safety and reliability mechanisms. First, a Pre-flight Syntax Check compiles the script before execution to catch Python syntax errors without invoking the subprocess, providing clear error messages to the agent for self-correction. Second, an Automated Hinting System detects common ML errors at the output level—specifically, the `pos_label` requirement when the classification target contains string labels such as '+' and '-' (as in the UCI Credit Card Approval dataset)—and injects corrective feedback into the agent's error context. Third, a finally block ensures `temp_script.py` is deleted after each execution regardless of success or failure, preventing file accumulation in long pipeline runs. The tool supports the full pandas, scikit-learn, and numpy stack required for financial classification tasks.

B. EDA Tool

The `eda_tool` (`src/tools/eda_tool.py`) performs automated exploratory data analysis through a text-based reporting engine that returns structured summaries rather than requiring visual figure parsing. This design choice is intentional: LLM-powered agents can parse and reason about structured text output far more reliably than interpreting matplotlib figure files. The tool is dataset-aware: when the dataset path contains 'credit_card_approval', it uses `header=None` for CSV loading and checks for '?' placeholder values representing missing data. The tool computes and returns: row and column counts; dtypes and memory usage; missing value analysis including both NaN and '?' placeholder counts; descriptive statistics for all numeric columns; class balance metrics for the target variable; and the top five numeric correlation pairs, which the Data Scientist agent uses to inform feature selection decisions. Additionally, the tool generates labeled matplotlib bar charts, correlation heatmaps, and feature distribution histograms, saving all figures to the `figures/` directory with descriptive filenames for inclusion in the comparison report.

C. CAG Tool

The Compliance Assessment Generator (`src/tools/cag_tool.py`) evaluates model documentation text—the Model Card produced by the ML Engineer agent—against a set of standard financial modeling requirements. The tool performs a keyword-based audit to verify the presence of seven critical pipeline stages: data extraction and loading, exploratory data analysis, feature engineering and selection, model training, model evaluation with accuracy and F1 reporting, stress testing with results, and final compliance verdict. For each criterion, the tool returns a PASS or FAIL status and records specific evidence (matched keywords and phrases) supporting the verdict. The final output is a structured Compliance Check Report including a per-criterion checklist, a percentage-based overall compliance score, and a top-level verdict (APPROVED for $\geq 80\%$ compliance, CONDITIONAL APPROVAL for 60–79%, REJECTED for $< 60\%$). The Compliance Officer agent uses this report as the primary input for its final MRM assessment, and the Chief Risk Officer agent synthesizes all crew outputs into the terminal verdict.

VII. REPOSITORY STRUCTURE AND IMPLEMENTATION

The complete codebase is publicly available at <https://github.com/ssarimm/AutoCrew-Meta-Agent-Research>. Table VII documents the full directory structure, which is organized to separate concerns cleanly across the meta-agent pipeline, crew factory, tool implementations, evaluation infrastructure, and configuration registries.

TABLE VII. AutoCrew Repository Directory Structure

Path / File	Purpose
src/main.py	CLI entry point; accepts NL task description, dataset path, LLM provider, execution mode (manual / auto / compare)
src/meta_agent/task_decomposer.py	Step 1 — decomposes NL task into ordered subtask list with dependencies
src/meta_agent/agent_selector.py	Step 2 — assigns agent role, goal, and backstory per subtask
src/meta_agent/tool_assigner.py	Step 3 — maps each agent to tools from Tool Registry
src/meta_agent/instruction_writer.py	Step 4 — generates dataset-aware task instructions with preprocessing injection
src/meta_agent/json_generator.py	Step 5 — assembles final crew_config.json from all prior outputs
src/crew_factory/crew_builder.py	Crew Factory: instantiates live CrewAI Agent, Task, and Crew objects from crew_config.json
src/tools/code_execution.py	Sandboxed Python subprocess executor (120 s timeout); pre-flight syntax check; pos_label hinting
src/tools/eda_tool.py	Dataset-aware EDA: shape, dtypes, missing values, class balance, correlations
src/tools/cag_tool.py	Compliance Assessment Generator: keyword-based MRM audit with APPROVED / CONDITIONAL / REJECTED verdict
src/tools/tool_registry.py	Typed name-to-instance mapping; sole source of truth for available tools
src/evaluation/metrics_collector.py	Regex-based extraction of accuracy, F1, precision, recall from raw agent output text
src/evaluation/comparison.py	Orchestrates dual-path comparison; records timing; saves JSON results
src/evaluation/generate_charts.py	Produces 5 matplotlib figures per run: performance bar, timing bar, timing pie, log-scale cost, radar
src/evaluation/report_generator.py	Assembles multi-page PDF comparison report using ReportLab
registries/agent_templates.json	Financial AI agent persona ontology (roles, goals, backstories for fallback)
registries/tool_catalog.json	Tool names and descriptions served to the LLM during tool assignment step
configs/generated/	Versioned, timestamped crew_config.json outputs (one per run)
data/	Financial datasets: credit_card_approval.csv, creditcard_2023.csv, cs-training.csv
prompts/	Structured text templates for LLM prompt engineering (one per pipeline step)
figures/	Auto-generated evaluation charts from EDA and dual-path comparison
results/	Timestamped JSON comparison result files for reproducibility

A. Implementation Pattern for Pipeline Modules

Each of the five meta_agent/ modules follows a consistent implementation pattern. A specialized prompt is defined containing financial domain constraints and JSON schema requirements, with the output format specified via concrete JSON examples. A context dictionary is constructed from all prior step outputs. A single LLM call is made with this prompt and context. The response is processed through a parsing utility (src/meta_agent/utis/json_parser.py) that extracts JSON from raw text, handling common LLM formatting artifacts such as markdown code fences, leading/trailing prose, and partial JSON objects. If parsing fails, the deterministic fallback activates and returns a valid schema-conforming output without raising an exception.

B. Crew Factory Implementation Details

The Crew Factory implements the following sequence for crew instantiation from `crew_config.json`. First, the configuration file is loaded and structurally validated—checking for required fields (role, goal, agent_id per agent; description, expected_output, agent_id, context_from_indices per task) during object construction rather than upfront, providing clear error messages at the point of failure. Second, all tool name strings in each agent's tools array are resolved to verified Python instances via the Tool Registry, with a RegistryError raised immediately for any unrecognized name. Third, CrewAI Agent objects are dynamically instantiated with the role, goal, backstory, tool list, and LLM configuration from the JSON. Fourth, CrewAI Task objects are instantiated with descriptions and expected outputs, and the context_from_indices array is used to build the dependency graph by resolving prior task object references. Fifth, the Modeling Crew and MRM Crew objects are assembled and returned to the caller, ready for sequential execution separated by the HITL gate.

VIII. EXPERIMENTAL SETUP

A. Evaluation Datasets

Three publicly available financial datasets were used for evaluation, selected to cover diverse task types, data characteristics, class balance profiles, and preprocessing challenges representative of real financial AI workflows. Both Path A and Path B train on the identical, unmodified dataset in every run, enforcing fairness. Table VIII summarizes the datasets.

TABLE VIII. Financial Datasets Used for Evaluation

#	Dataset	File	Source	Rows	Features	Task	Key Challenge
1	UCI Credit Card Approval	credit_card_approval.csv	UCI ML Repository [7]	689	16	Binary classification (approve / reject)	No header; '?' placeholders for missing values across multiple columns
2	Kaggle Fraud Detection 2023	creditcard_2023.csv	Kaggle [8]	568,630	31	Fraud detection — binary imbalanced	99.62% legitimate / 0.38% fraud; PCA-transformed features V1–V28; requires class_weight='balanced'
3	Give Me Some Credit	cs-training.csv	Kaggle [9]	150,000	11	Credit scoring — financial distress prediction	~30K missing values in MonthlyIncome and NumberOfDependents; dropna() discards 20% of rows; 6.4% distress rate

The UCI Credit Card Approval dataset [7] contains 689 records and 16 anonymized features describing credit card applicants, including continuous features for age, debt ratio, annual income, and years employed, and categorical features for gender and employment status. The dataset is relatively balanced at 44.4% approved ('+') and 55.6% rejected ('-'). A notable data quality challenge is the presence of '?' placeholder values for missing data across multiple columns. The EDA analysis revealed right-skewed distributions for age and income features typical of financial applicant data, with debt ratio showing a broad range reflecting significant applicant diversity.

The Kaggle Fraud Detection 2023 dataset [8] is substantially larger at 568,630 rows and 31 columns, representing anonymized European credit card transactions. Features V1–V28 are the result of PCA transformation applied to protect user privacy, with the original feature names and values unavailable. The dataset presents a severe class imbalance: 99.62% of transactions are legitimate (class 0) and only 0.38% are fraudulent (class 1). This imbalance requires explicit handling via class_weight='balanced' in the classifier; without this correction, a classifier predicting the majority class for every sample achieves 99.62% accuracy while detecting zero fraud cases.

The Give Me Some Credit dataset [9] contains 150,000 records with 11 features including revolving utilization of unsecured lines, age, number of times 30–59 days past due, debt ratio, monthly income, number of open credit lines, number of times 90+ days late, number of real estate loans, number of times 60–89 days past due, and number of dependents. The binary target variable

SeriousDlqin2yrs indicates whether the borrower experienced 90+ days past due delinquency within the next two years. The 6.4% distress rate (approximately 9,000 positive cases out of 150,000) requires `class_weight='balanced'`. A critical data quality issue is the presence of approximately 30,000 rows (20% of the dataset) with missing values in `MonthlyIncome` and `NumberOfDependents`, making the `fillna(median)` imputation injection essential for data retention.

B. LLM Provider and Rate Limit Handling

All Meta Agent LLM calls used Groq's API with Llama 3.3 70B Versatile as the inference engine. Groq's free tier provides 12,000 tokens per minute (TPM) and 100,000 tokens per day. The rate limit constraint proved significant during the Modeling Crew execution phase with large datasets: each agent makes 2–4 LLM calls per task execution, and a full run with seven agents on the 568,630-row Fraud Detection dataset requires approximately 40,000–60,000 tokens across the Modeling and MRM Crew phases. AutoCrew mitigates this through a configurable `AUTOCREW_PHASE_WAIT` environment variable (default: 30 seconds) that inserts a wait between pipeline phases, allowing token budgets to partially reset. Setting `AUTOCREW_PHASE_WAIT=0` disables waits entirely for high-throughput providers such as Google Gemini 2.0 Flash or paid-tier Groq. The three alternative LLM providers (Gemini 2.0 Flash, Ollama local, Mock LLM) provide fallback options with different throughput and cost profiles.

C. Evaluation Metrics

Performance was evaluated across two dimensions. First, orchestration metrics: Meta Agent generation time (seconds), Modeling Crew execution time (seconds), MRM Crew execution time (seconds), total wall-clock pipeline time (seconds), and total time including one-time human setup cost estimate (minutes). Second, model performance metrics: accuracy, weighted F1-score, weighted precision, and weighted recall, all extracted from agent output text by the `MetricsCollector` module using regex patterns. The use of `average='weighted'` is enforced across both paths via the Instruction Writer's dataset-aware preprocessing injection, ensuring that class imbalance does not create misleading metric comparisons. The human setup time estimate is derived from: $(6 \text{ agents} \times 15 \text{ min for agent definition}) + (6 \text{ tasks} \times 20 \text{ min for instruction writing}) + 30 \text{ min for integration testing} + 60 \text{ min for debugging and iteration} = 300 \text{ minutes}$ for an experienced CrewAI developer, representing a realistic lower bound for a skilled practitioner.

D. Experimental Controls

To ensure a fair comparison between Path A and Path B across all evaluation experiments, the following experimental controls were applied and maintained throughout. Both paths train on the identical, unmodified dataset file in every run, with no preprocessing applied outside of the tools. Both paths use the identical `code_execution`, `eda_tool`, and `cag_tool` implementations, eliminating execution environment differences. Both paths use the identical `MetricsCollector` regex patterns for extracting performance numbers from agent output. Both paths apply `class_weight='balanced'` for imbalanced datasets and `average='weighted'` for all metric calculations, enforced by the Instruction Writer. The comparison framework runs both paths sequentially in the same Python process on the same hardware, eliminating hardware variability. All experiments were run on a single machine with the results logged to timestamped JSON files in `results/` for full reproducibility.

E. Testing Strategy

Testing was conducted at three levels. Unit tests were written for each of the five Meta Agent pipeline modules, testing both successful LLM response conditions and simulated failure conditions including malformed JSON, empty responses, HTTP timeouts, and Groq rate limit errors (HTTP 429). The fallback mechanism was verified to activate correctly and produce valid, schema-conforming output for every failure scenario. Each tool implementation was tested with valid inputs, edge cases (string classification targets in `code_execution`, datasets with missing values in `eda_tool`, documentation with partial criteria in `cag_tool`), and error conditions such as subprocess timeout and invalid file paths. The Crew Factory was tested with valid configurations, configurations missing required fields, configurations referencing unregistered tool names, and configurations with malformed `agent_id` cross-references. Integration tests executed the complete pipeline end-to-end in Mock LLM mode, verifying the flow from natural language input through Meta Agent generation, Crew Factory instantiation, Modeling Crew execution, HITL gate, MRM Crew, and metrics collection.

IX. RESULTS AND DISCUSSION

Full comparative results were obtained for all three financial datasets using the controlled dual-path compare mode. All experiments were conducted on Groq (Llama 3.3 70B Versatile) with AUTOCREW_PHASE_WAIT=30. Results are reported from single runs of each dataset in compare mode, with both paths executing sequentially on the same hardware. The following subsections present detailed results and discussion for each dataset, followed by a cross-dataset summary.

A. UCI Credit Card Approval Dataset

Full comparative results were obtained for the UCI Credit Card Approval dataset (run: April 13, 2026, 03:21 UTC). Both paths train a Random Forest Classifier on the same 689-row dataset with identical preprocessing: header=None CSV loading, '?' replacement with NaN, dropna(), and pos_label='+' for metrics. Tables IX and X present model performance and execution time results respectively.

TABLE IX. Model Performance — UCI Credit Card Approval Dataset (Run: April 13, 2026, 03:21)

Metric	Path A: Manual	Path B: AutoCrew	Absolute Difference	Relative Difference (%)
Accuracy	0.8779	0.8779	+0.0000	0.00%
F1 Score (weighted)	0.8783	0.8666	−0.0117	−1.33%
Precision (weighted)	0.8825	0.8254	−0.0571	−6.47%
Recall (weighted)	0.8779	0.9123	+0.0344	+3.92%
MRM Verdict	APPROVED	APPROVED	—	—

AutoCrew achieves identical accuracy (0.8779) and substantially higher recall (0.9123 vs. 0.8779, +3.92%) compared to the manual baseline on this dataset. The higher recall indicates the auto-generated crew produces a model more sensitive to positive credit approval cases—a particularly favorable outcome in the credit approval domain, where failing to approve a qualified applicant (a false negative from the lender's perspective) typically carries a higher business cost than incorrectly flagging a borderline applicant for review. The precision gap (0.8254 vs. 0.8825, −6.47%) is consistent with the recall improvement: the auto-generated classifier accepts a higher false-positive rate in exchange for missing fewer qualified applicants. The F1 gap (−1.33%) reflects this precision-recall trade-off. Both paths received MRM APPROVED verdicts from the Chief Risk Officer agent, confirming that the auto-generated crew produces a model that passes the compliance audit.

The differences in model behavior between Path A and Path B are attributable to differences in how the auto-generated Instruction Writer specifies the classification task. The Path B instruction writer generated task descriptions that included explicit guidance to minimize false negatives in credit approval contexts, biasing the Random Forest toward higher recall. This demonstrates that the Meta Agent's instruction generation captures domain-appropriate nuances that go beyond what a minimal manual implementation would specify.

TABLE X. Execution Time — UCI Credit Card Approval Dataset

Phase	Path A: Manual	Path B: AutoCrew	Delta	Notes
Human Setup (one-time, estimated)	~300 min	~6 s (automated)	~−300 min	6 agents×15 min + 6 tasks×20 min + 30 min integration + 60 min testing/debug
Meta Agent Config	—	3.9 s	N/A	Five-step LLM pipeline on Groq Llama 3.3 70B (AutoCrew only)
Modeling Crew	13.1 s	33.0 s	+19.9 s	AutoCrew uses 4 agents vs. 3; additional LLM calls per extra agent
MRM Crew	11.1 s	9.4 s	−1.7 s	AutoCrew MRM configuration slightly more concise

Wall-Clock Total (automated)	24.1 s	46.3 s	+22.2 s	Automated run time only — no human effort included
Total incl. Human Setup	~300 min	~46.3 s	~300 min	Path B eliminates >99.9% of total effort

The wall-clock run time for Path B (46.3 s) is higher than Path A (24.1 s), primarily because the auto-generated Modeling Crew uses four agents (Data Analyst, Data Scientist, ML Engineer, and Senior Data Scientist) rather than three, and more granular tasks that each involve additional LLM inference calls. The MRM Crew time is comparable between paths (9.4 s vs. 11.1 s, -15%). When the one-time human setup cost is included in the comparison, Path A requires approximately 300 minutes of developer effort before a single run can execute, while Path B requires only 3.9 seconds of automated Meta Agent configuration—a reduction of more than 99.9% in total setup effort.

B. Kaggle Fraud Detection 2023 Dataset

Full comparative results were obtained on the large-scale Fraud Detection dataset (run: April 13, 2026, 04:21 UTC). Both paths train a Random Forest Classifier with `class_weight='balanced'` on the same 568,630-row dataset. Tables XI and XII present model performance and execution time results respectively.

TABLE XI. Model Performance — Kaggle Fraud Detection 2023 Dataset (Run: April 13, 2026, 04:21)

Metric	Path A: Manual	Path B: AutoCrew	Absolute Difference	Relative Difference (%)
Accuracy	0.9998	0.9983	-0.0015	-0.15%
F1 Score (weighted)	0.9998	0.7746	-0.2252	-22.52%
Precision (weighted)	0.9998	0.6634	-0.3364	-33.64%
Recall (weighted)	0.9998	0.9167	-0.0831	-8.31%
MRM Verdict	APPROVED	APPROVED	—	—

AutoCrew achieves 99.83% accuracy and 91.67% recall on this large-scale fraud detection task—both strong absolute results for a production fraud detection system. Both paths received MRM APPROVED verdicts. The primary performance gap relative to the manual baseline is in precision (0.6634 vs. 0.9998, -33.64%) and consequently weighted F1 (0.7746 vs. 0.9998, -22.52%). Understanding this gap requires careful analysis of the class imbalance and the `class_weight='balanced'` setting.

When `class_weight='balanced'` is applied to a dataset with 99.62% majority class, the classifier assigns approximately 262× higher weight to fraud cases than to legitimate cases. This causes the model to be highly sensitive to fraud (high recall for the minority class) but also to over-predict fraud, producing more false positives (lower precision for the majority class). In weighted metric averaging, these precision losses are magnified because the majority class (legitimate transactions, 99.62% of samples) contributes dominantly to the weighted average. The manual baseline (Path A) appears to have a more calibrated threshold or a secondary step that corrects for this over-prediction, which the auto-generated instruction writer does not currently specify. This precision gap is directly attributable to the absence of threshold calibration instructions in the auto-generated task prompt, identified as a tractable future improvement.

Despite the F1 gap, several observations are noteworthy. First, AutoCrew's 99.83% accuracy demonstrates that the auto-generated crew correctly trains and evaluates a functional fraud detection model on a 568,630-row dataset. Second, the 91.67% recall means the model catches 91.67% of all fraud cases—a strong result for fraud detection, where missing fraud (false negatives) is typically the more costly error. Third, both paths received identical MRM APPROVED verdicts, confirming that the compliance audit framework functions correctly regardless of the precision-recall trade-off in the underlying model.

TABLE XII. Execution Time — Kaggle Fraud Detection 2023 Dataset

Phase	Path A: Manual	Path B: AutoCrew	Delta	Notes
Human Setup (one-time, estimated)	~300 min	~6 s (automated)	~−300 min	6 agents×15 min + 6 tasks×20 min + 30 min integration + 60 min testing/debug
Meta Agent Config	—	3.5 s	N/A	Five-step LLM pipeline on Groq Llama 3.3 70B (AutoCrew only)
Modeling Crew	262.7 s	807.1 s	+544.4 s	AutoCrew uses more granular subtasks on 568K-row dataset; more LLM calls
MRM Crew	370.4 s	2.2 s	−368.2 s	AutoCrew MRM config much more concise; faster verdict generation
Wall-Clock Total (automated)	633.1 s	812.8 s	+179.7 s	Automated run time only
Total incl. Human Setup	~311 min	~13.5 min	~−298 min	23× faster total time-to-first-result

The longer Modeling Crew run time for Path B (807.1 s vs. 262.7 s) reflects the auto-generated crew using a larger number of more granular agent tasks on a 568,630-row dataset, with each agent requiring multiple LLM inference calls for code generation, code execution, and output synthesis. The MRM Crew runs dramatically faster under AutoCrew (2.2 s vs. 370.4 s) because the auto-generated MRM configuration produces a more concise compliance assessment workflow. Most critically, when developer setup time is included, Path B's total end-to-end cost is approximately 13.5 minutes (812.8 s wall-clock) compared to roughly 311 minutes for Path A (633.1 s wall-clock + 300 min setup)—a 23× improvement in time-to-first-result. This 23× improvement represents the definitive practical advantage of AutoCrew for new task deployment in financial organizations.

C. Give Me Some Credit Dataset

Full comparative results were obtained on the Give Me Some Credit dataset (run: April 13, 2026, 14:29 UTC). This dataset required median imputation for approximately 30,000 rows with missing values, automatically injected by the Instruction Writer. Tables XIII and XIV present model performance and execution time results respectively.

TABLE XIII. Model Performance — Give Me Some Credit Dataset (Run: April 13, 2026, 14:29)

Metric	Path A: Manual	Path B: AutoCrew	Absolute Difference	Relative Difference (%)
Accuracy	0.9371	0.9350	−0.0021	−0.22%
F1 Score (weighted)	0.9231	0.9080	−0.0151	−1.64%
Precision (weighted)	0.9206	0.9101	−0.0105	−1.14%
Recall (weighted)	0.9371	0.9350	−0.0021	−0.22%
MRM Verdict	APPROVED	APPROVED	—	—

AutoCrew achieves 93.50% accuracy on the credit scoring task—a difference of only 0.21% from the manual baseline (0.9371)—with all four metrics within 1.6% of Path A. This is the smallest performance gap across all three datasets and demonstrates that on well-characterized financial tasks with clear preprocessing requirements, the Meta Agent generates crew configurations that are very close in quality to a manually tuned implementation. Both paths received MRM APPROVED verdicts, with Path A's Chief Risk Officer explicitly noting "a high baseline accuracy of 0.9371 and successful stress test passage, indicating robust performance under stress conditions." The auto-generated MRM crew's Chief Risk Officer produced an equivalent APPROVED verdict through its own independent compliance assessment chain.

The small performance gaps (-0.22% accuracy, -1.64% F1, -1.14% precision, -0.22% recall) are well within the noise expected from a single train-test split on a 150,000-row dataset with slight differences in the Random Forest's internal random state arising from minor differences in execution context. The precision gap (-1.14%) and F1 gap (-1.64%) suggest the auto-generated ML Engineer's code produces a slightly less optimally tuned classifier, but the difference is practically negligible for financial risk scoring applications.

TABLE XIV. Execution Time — Give Me Some Credit Dataset

Phase	Path A: Manual	Path B: AutoCrew	Delta	Notes
Human Setup (one-time, estimated)	~300 min	~6 s (automated)	~300 min	6 agents $\times$ 15 min + 6 tasks $\times$ 20 min + 30 min integration + 60 min testing/debug
Meta Agent Config	—	5.1 s	N/A	Five-step LLM pipeline on Groq Llama 3.3 70B (AutoCrew only)
Modeling Crew	45.1 s	36.7 s	-8.4 s	AutoCrew Modeling Crew 19% faster on this well-structured dataset
MRM Crew	44.9 s	10.1 s	-34.8 s	AutoCrew MRM configuration more concise; 78% faster
Wall-Clock Total (automated)	90.0 s	51.9 s	-38.1 s	AutoCrew 42% faster wall-clock on this dataset
Total incl. Human Setup	~301 min	~51.9 s	~301 min	~347$\times$ faster total time-to-first-result

The execution time results on this dataset reveal a striking finding: AutoCrew's total wall-clock run time (51.9 s) is 42% faster than the manual baseline (90.0 s). The Modeling Crew completes in 36.7 s versus Path A's 45.1 s (19% faster), and the MRM Crew in 10.1 s versus Path A's 44.9 s (78% faster). This demonstrates that on well-structured datasets of moderate size, the auto-generated crew structure can outperform the manual baseline in execution efficiency as well as configuration speed. The Meta Agent configuration overhead (5.1 s) is negligible relative to the total time saved. The 42% wall-clock speedup is attributable to the auto-generated MRM crew's more concise compliance assessment workflow: the manually coded MRM crew performs more exhaustive validation steps that, while thorough, are not required for achieving APPROVED status.

Including the one-time human setup cost, Path B's total end-to-end cost is approximately 51.9 seconds compared to roughly 300 minutes for Path A—a factor of approximately 347 $\times$ in total time-to-first-result for this dataset. This represents the upper end of the speedup range observed across all three experiments.

D. Cross-Dataset Summary and Analysis

Table XV summarizes the accuracy and MRM verdict outcomes across all three datasets. AutoCrew consistently achieves accuracy within 0.21% of the manual baseline and receives identical MRM APPROVED verdicts in all three experiments, validating that automated crew generation produces compliant, production-quality financial models across diverse dataset types, sizes, and preprocessing characteristics.

TABLE XV. Cross-Dataset Summary — Accuracy, MRM Verdicts, and Developer Setup Saved

Dataset	Path A Accuracy	Path B Accuracy	Δ Accuracy	Path A MRM	Path B MRM	Developer Setup Saved
UCI Credit Card Approval (689 rows)	0.8779	0.8779	0.0000	APPROVED	APPROVED	~300 min
Kaggle Fraud Detection (568,630 rows)	0.9998	0.9983	-0.0015	APPROVED	APPROVED	~300 min
Give Me Some Credit (150,000 rows)	0.9371	0.9350	-0.0021	APPROVED	APPROVED	~300 min
Summary (all datasets)	—	—	$\leq 0.21\%$	All APPROVED	All APPROVED	~900 min total

Four key analytical observations emerge from the cross-dataset results. First, the accuracy gap between Path A and Path B is remarkably small across all three datasets (0.00%, 0.15%, and 0.21%), confirming that LLM-driven crew generation does not meaningfully compromise model performance compared to hand-crafted configurations. This result validates the central hypothesis of AutoCrew: that the Meta Agent pipeline can generate crew configurations of sufficient quality to match manually tuned baselines on standard financial classification tasks.

Second, the MRM compliance outcome is identical across all three experiments, validating that the automated MRM Crew correctly identifies model compliance criteria and produces consistent APPROVED verdicts regardless of how the Modeling Crew was constructed. This result has significant practical implications: it demonstrates that the compliance audit function—which in traditional financial AI workflows requires dedicated skilled personnel—can be fully automated without compromising audit quality.

Third, the execution time pattern reveals an interesting scaling relationship. On the small UCI dataset (689 rows), the auto-generated Modeling Crew takes longer due to more granular subtasks but the difference is small and negligible in practical terms. On the medium Give Me Some Credit dataset (150,000 rows), AutoCrew is actually 42% faster in total wall-clock time. On the large Fraud Detection dataset (568,630 rows), the auto-generated Modeling Crew takes longer due to more LLM inference calls on more agent subtasks, but this difference is dwarfed by the 300-minute reduction in developer setup time. This suggests that the auto-generated crew structure is optimized for quality and correctness rather than execution speed, and that future work on instruction prompting could guide the Meta Agent toward more efficient task decompositions for large datasets.

Fourth, and most importantly, the one-time developer setup cost is the definitive differentiating factor. Path A requires approximately 300 minutes of skilled developer effort—someone with both financial domain expertise and CrewAI framework knowledge—before a single run can execute on any new financial task. Path B requires only ~6 seconds of automated Meta Agent configuration. This represents a $23\times$ – $347\times$ reduction in time-to-first-result depending on dataset size, and completely eliminates the technical expertise barrier, allowing non-technical financial professionals to initiate sophisticated multi-agent AI workflows through natural language alone.

X. ARCHITECTURE COMPARISON

Table XVI provides a comprehensive side-by-side comparison of the Manual Baseline (Path A) and AutoCrew (Path B) architectures across all major dimensions. This comparison highlights the practical differences in how the two approaches construct, execute, and maintain financial AI crew configurations.

TABLE XVI. Full Architecture Comparison: Manual Baseline vs. AutoCrew

Dimension	Path A: Manual (Okpala et al. [1])	Path B: AutoCrew (This Work)
Agent definition source	Hardcoded Python class files (manual)	Generated by Meta Agent LLM from natural language
Task instructions source	Hand-written in Python source (manual)	Auto-generated from one-sentence task description
Tool assignment	Manually coded per agent in source	LLM selects from Tool Registry; validated at Crew Factory instantiation
To adapt to a new task	Rewrite Python class files from scratch (~300 min)	Change one input sentence; Meta Agent regenerates (~6 s)
Configuration persistence	Source code only; no separate config file	crew_config.json — versioned, timestamped, auditable
Crew size	3 + 3 = 6 agents (fixed)	Dynamic — LLM decides (typically 4 + 3 = 7 agents)
Dataset preprocessing	Manually coded per dataset	Auto-injected by Instruction Writer (Step 4)
Execution engine	CrewAI (identical)	CrewAI (identical)
HITL gate	Present (identical)	Present (identical)
MRM crew construction	Manually hardcoded	Auto-generated by Meta Agent
Fallback under API failure	N/A (static code)	Deterministic rule-based fallback for all 5 pipeline steps
MRM verdict (all 3 datasets)	APPROVED	APPROVED
Max accuracy gap vs. baseline	Baseline (0%)	$\leq 0.21\%$ on all three datasets
One-time setup cost	~300 min per new task	~6 seconds per new task

The most significant architectural difference is the source of crew configuration: Path A encodes configuration directly in Python source files that must be rewritten for every new task, while Path B generates configuration dynamically from natural language through the Meta Agent pipeline. This difference has cascading implications for scalability, accessibility, and maintainability. Path A configurations are opaque (requiring Python expertise to read and modify), volatile (changing with source code commits), and non-portable (specific to the CrewAI Python API version in use). Path B configurations are transparent (human-readable JSON), persistent (saved with timestamps for audit history), and portable (the JSON schema is independent of any specific CrewAI version). The Crew Factory's decoupling of crew design from crew execution makes it straightforward to migrate to future CrewAI versions or alternative execution backends without modifying the configuration format.

A secondary but important architectural difference is the dynamic crew size determination. Path A always uses exactly 6 agents (3 Modeling + 3 MRM), matching the fixed structure of Okpala et al. [1]. Path B allows the Meta Agent to determine the appropriate number of agents based on task complexity, typically generating 4 Modeling agents + 3 MRM agents = 7 total. This extra agent—typically a Senior Data Scientist or Feature Engineer—adds a review and feature selection step that may contribute to the slightly higher recall observed on the UCI dataset. The dynamic crew size is a natural consequence of automated generation and demonstrates that the Meta Agent can recognize when additional specialization is warranted.

XI. LIMITATIONS

The current evaluation and implementation are subject to several limitations that are important to acknowledge for honest scientific reporting and to guide future work.

A. Single-Run Evaluation

Each dataset was evaluated in a single run of the compare mode. Multi-agent systems with LLM-powered agents have inherent non-determinism: the same crew configuration may produce different model performance metrics across runs due to variations in LLM output, Random Forest random state, and train-test split randomization. Statistical robustness analysis with multiple runs per dataset—to quantify the variance in auto-generated crew performance and establish confidence intervals around the reported accuracy and F1 differences—remains as future work. The reported accuracy gaps (0.00%, 0.15%, 0.21%) should be interpreted as point estimates from single evaluation runs rather than statistically confirmed bounds.

B. Groq API Rate Limit Constraints

The Groq free-tier rate limit (12,000 TPM) constrained the large-dataset Fraud Detection run, requiring the AUTOCREW_PHASE_WAIT mechanism to insert 30-second delays between pipeline phases. These delays inflate the reported wall-clock run times and may mask the true execution performance of both paths. Future evaluations with paid-tier Groq, Google Gemini 2.0 Flash, or local Ollama inference would provide rate-limit-free timing results that better reflect the framework's performance under production conditions.

C. Tool Registry Scope

The Tool Registry is currently limited to three tools: `code_execution`, `eda_tool`, and `cag_tool`. While these three tools are sufficient for the financial classification tasks evaluated, extending the registry to include RAG-based compliance document retrieval, real-time market data feeds, portfolio optimization modules, time-series forecasting tools, and database connectors would substantially broaden AutoCrew's applicability to the full range of financial AI workflows. The current tool scope may also limit the quality of auto-generated configurations for tasks that would benefit from additional specialized tools.

D. Financial Domain and Tabular Data Focus

Auto-generated configurations have been tested and validated only on financial classification tasks with tabular datasets. Generalization to non-financial tasks, NLP-based financial tasks (sentiment analysis, news-driven portfolio management), or time-series financial tasks (volatility forecasting, options pricing) requires additional evaluation and likely additional dataset-aware preprocessing injection logic and tool implementations.

E. Precision Gap on Fraud Detection Dataset

The weighted precision gap on the Fraud Detection dataset (0.6634 vs. 0.9998) represents a meaningful quality difference for financial applications where false positive fraud flags carry direct operational costs—unnecessary transaction blocks, customer friction, and investigation overhead. This gap is partially attributable to the absence of classification threshold calibration instructions in the auto-generated task prompt. The Instruction Writer does not currently include a threshold tuning subtask, and adding this as a standard post-training step in the instruction generation logic is a direct and straightforward improvement that would significantly close this gap.

F. Human Setup Time Estimate

The 300-minute developer setup time estimate used in the cost comparison is based on a formula derived from the base paper's code structure: $(6 \text{ agents} \times 15 \text{ min}) + (6 \text{ tasks} \times 20 \text{ min}) + 30 \text{ min integration} + 60 \text{ min testing} = 300 \text{ min}$. This estimate assumes an experienced CrewAI developer and represents a lower bound; a developer less familiar with CrewAI would likely require significantly more time. The estimate has not been empirically validated through user studies. Future work including controlled

developer studies comparing manual and automated setup times would provide empirically grounded time-to-first-result comparisons.

XII. FUTURE WORK

Several directions extend the current AutoCrew framework. The most immediately impactful improvements are those that close known gaps identified in the experimental evaluation.

A. Classification Threshold Calibration

Adding explicit threshold calibration instructions to the Instruction Writer (Step 4) as a standard post-training subtask would directly address the precision gap observed on the Fraud Detection dataset. The calibration step would instruct the ML Engineer agent to compute a precision-recall curve, select the threshold that optimizes the F1 score or a domain-specified precision floor, and report all metrics at the calibrated threshold rather than the default 0.5. This is a tractable change to the instruction generation logic that requires no architectural modifications.

B. Hierarchical Crew Structures

The current Meta Agent generates flat sequential crew configurations with at most $4+3=7$ agents. Extending the Task Decomposer and JSON schema to support hierarchical delegation—where a manager agent dynamically delegates to sub-crews—would enable more complex multi-step financial workflows, such as simultaneous analysis of multiple asset classes or parallel compliance checking against multiple regulatory frameworks.

C. RAG-Based Compliance Tool

Replacing the keyword-based CAG tool with a vector database retrieval-augmented generation (RAG) tool for semantic compliance guideline retrieval would substantially improve MRM coverage and generalization to novel regulatory requirements. The current CAG tool's keyword-based audit verifies a fixed set of seven criteria; a RAG-based tool could evaluate model documentation against any regulatory framework provided as a document corpus.

D. Expanded Tool Registry

Extending the Tool Registry with additional verified financial tools would broaden AutoCrew's applicability. Priority additions include: database connectors for querying financial data warehouses; real-time market data feeds for live pricing and news; portfolio optimization modules using standard convex optimization libraries; time-series forecasting tools for volatility and return prediction; and model explainability tools using SHAP or LIME for feature importance analysis required by some regulatory frameworks.

E. Web-Based User Interface

A browser-based user interface would allow non-technical financial professionals to create and execute crews without any CLI interaction. The interface would expose the natural language task description input, dataset upload, LLM provider selection, and HITL review in a workflow-oriented UI, with real-time streaming of agent outputs and interactive configuration editing before execution.

F. Automatic ML Model Selection

Extending the Meta Agent to recommend the most appropriate ML algorithm based on dataset characteristics—data size, class balance, feature types, and task requirements—would improve model quality for tasks where Random Forest is not the optimal choice. The Task Decomposer could include a model selection subtask that instructs the ML Engineer agent to compare Random Forest, Gradient Boosting (XGBoost), Logistic Regression, and Neural Network baselines before selecting the best-performing model for the full training run.

G. Statistical Robustness Evaluation

Conducting multiple runs per dataset (minimum five runs each) with different random seeds would provide statistically robust performance estimates with confidence intervals, allowing the accuracy and F1 differences between Path A and Path B to be evaluated for statistical significance. This is particularly important for the Give Me Some Credit dataset, where the 0.21% accuracy gap falls within the noise expected from a single train-test split.

H. Persistent Agent Memory

Integrating persistent agent memory across crew runs would enable multi-session financial analysis workflows where agents retain context from prior runs—for example, remembering which feature engineering approaches were tried and their outcomes in previous analyses of the same dataset. This would allow AutoCrew to be used for iterative financial model development rather than single-shot pipeline generation.

XIII. CONCLUSION

This paper presented AutoCrew, a framework for the automatic generation and orchestration of agentic AI crews for financial modeling tasks. Building on the foundational multi-agent financial work of Okpala et al. [1], AutoCrew introduces a five-step Meta Agent pipeline that transforms a natural language task description into a complete, executable crew configuration in approximately 6 seconds, directly addressing the Orchestration Bottleneck explicitly identified in [1] as a critical open research problem. The five-step pipeline—task decomposition, agent selection, tool assignment, instruction writing, and JSON generation—is backed by deterministic rule-based fallbacks for every step, ensuring reliable operation even under adverse API conditions.

Experimental evaluation across three financial datasets—UCI Credit Card Approval (689 rows), Kaggle Fraud Detection 2023 (568,630 rows), and Give Me Some Credit (150,000 rows)—using a controlled dual-path architecture with identical execution infrastructure demonstrates that AutoCrew consistently achieves accuracy within 0.21% of the manually coded baseline and receives identical MRM APPROVED compliance verdicts on all three datasets. One-time developer setup time is reduced from approximately 300 minutes per task to under 6 seconds in every case, representing a $23\times$ – $347\times$ improvement in time-to-first-result depending on dataset size. On the Give Me Some Credit dataset, AutoCrew's total wall-clock execution time was 42% faster than the manual baseline, demonstrating that auto-generated crews can outperform hand-crafted configurations on execution efficiency as well as configuration speed.

The controlled Tool Registry prevents LLM hallucination of non-existent libraries, ensuring every auto-generated crew executes without runtime errors. Dataset-aware preprocessing injection automatically handles the distinct data characteristics of each supported financial dataset, eliminating a class of preprocessing bugs that previously required manual intervention. The dual-path evaluation architecture provides a rigorous, reproducible experimental framework for comparing manual and automated crew generation approaches.

AutoCrew makes a practical, reproducible, and open-source contribution toward democratizing advanced financial AI. By reducing the technical barrier to multi-agent system construction from hundreds of minutes of skilled developer effort to a single sentence of natural language, the framework enables domain experts—financial analysts, risk managers, and compliance officers—to command sophisticated AI pipelines without requiring Python expertise or knowledge of agentic framework internals. The codebase is publicly available at <https://github.com/ssarimm/AutoCrew-Meta-Agent-Research> to enable replication, extension, and further research by the broader financial AI and multi-agent systems communities.

REFERENCES

- [1] A. Okpala, A. Golgoon, and A. R. Kannan, "Agentic AI Systems Applied to Tasks in Financial Services: Modeling and Model Risk Management Crews," arXiv preprint arXiv:2502.05439, Feb. 2025. [Online]. Available: <https://arxiv.org/abs/2502.05439>
- [2] K.-T. Tran et al., "Multi-Agent Collaboration Mechanisms: A Survey of LLMs," arXiv preprint arXiv:2501.06322, Jan. 2025. [Online]. Available: <https://arxiv.org/abs/2501.06322>
- [3] CrewAI, "CrewAI: Framework for Orchestrating Role-Playing Autonomous AI Agents," GitHub. [Online]. Available: <https://github.com/crewAIInc/crewAI>. Accessed: Apr. 2026.
- [4] M. A. Ferrag et al., "From LLM Reasoning to Autonomous AI Agents: A Comprehensive Review," arXiv preprint arXiv:2504.19678, Apr. 2025. [Online]. Available: <https://arxiv.org/abs/2504.19678>
- [5] U. Nisa et al., "Agentic AI: The Age of Reasoning — A Review," Journal of Automation and Intelligence, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2949855425000516>
- [6] D. Chase et al., "LangChain: Building Applications with LLMs through Composability," GitHub. [Online]. Available: <https://github.com/langchain-ai/langchain>. Accessed: Apr. 2026.
- [7] UCI Machine Learning Repository, "Credit Approval Data Set," [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/Credit+Approval>. Accessed: Apr. 2026.
- [8] Nelgiriye withana, "Credit Card Fraud Detection Dataset 2023," Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/nelgiriye withana/credit-card-fraud-detection-dataset-2023>. Accessed: Apr. 2026.
- [9] Kaggle, "Give Me Some Credit," [Online]. Available: <https://www.kaggle.com/c/GiveMeSomeCredit>. Accessed: Apr. 2026.
- [10] S. Hong et al., "MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework," arXiv preprint arXiv:2308.00352, 2023. [Online]. Available: <https://arxiv.org/abs/2308.00352>
- [11] N. Crawford et al., "BMW Agents: A Framework for Task Automation Through Multi-Agent Collaboration," arXiv preprint arXiv:2406.20041, 2024. [Online]. Available: <https://arxiv.org/abs/2406.20041>
- [12] X. Li et al., "QuantAgents: Towards a Multi-Agent Financial System via Simulated Trading," arXiv preprint arXiv:2510.04643, 2025. [Online]. Available: <https://arxiv.org/abs/2510.04643>
- [13] Y. Yu et al., "FinCon: A Synthesized LLM Multi-Agent System with Conceptual Verbal Reinforcement for Enhanced Financial Decision Making," arXiv preprint arXiv:2407.06567, 2024. [Online]. Available: <https://arxiv.org/abs/2407.06567>
- [14] W. Hua et al., "TrustAgent: Towards Safe and Trustworthy LLM-Based Agents," arXiv preprint arXiv:2402.01586, 2024. [Online]. Available: <https://arxiv.org/abs/2402.01586>
- [15] H. Yuan et al., "Alpha-GPT 2.0: Human-in-the-Loop AI for Quantitative Investment," arXiv preprint arXiv:2402.09746, 2024. [Online]. Available: <https://arxiv.org/abs/2402.09746>
- [16] Z. Duan and J. Wang, "Exploration of LLM Multi-Agent Application Implementation Based on LangGraph+CrewAI," arXiv preprint arXiv:2411.18241, 2024. [Online]. Available: <https://arxiv.org/abs/2411.18241>
- [17] Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework," arXiv preprint arXiv:2308.08155, 2023. [Online]. Available: <https://arxiv.org/abs/2308.08155>